DSA REFERENCE

Heap and Priority Queue Failure Modes

Common wrong turns, debugging cues, and complexity traps.

Keep the next best candidate accessible while updates cost logarithmic time instead of repeatedly sorting everything.

VERSION 1.0 REVIEW

REVIEWED THROUGH 2026-07-18

STATUS HUMAN REVIEW REQUIRED

DSA / HEAP-PRIORITY-QUEUE

DSA

Heap and Priority Queue Failure Modes

CORE IDEA

Keep the next best candidate accessible while updates cost logarithmic time instead of repeatedly sorting everything.

Recognition cues

+ top k
+ streaming minimum or maximum
+ merge sorted inputs
+ schedule next event

Implementation sequence

+ choose min-heap or max-heap
+ store tie-breakers
+ bound heap size for top k
+ remove stale entries when using lazy deletion

Invariant

The root is the best element under the chosen ordering; heap contents are exactly the active candidates.

Worked reasoning

For top k largest values, keep a min-heap of size k; its root is the cutoff.

Complexity

O(log k) per bounded update and O(k) space; full heap construction can be O(n).

Failure modes

- [ ] reversing heap direction
- [ ] unstable tuple comparisons
- [ ] forgetting stale entries
- [ ] sorting when incremental answers are required

Practice ladder

Heap and Priority Queue Failure Modes: referenced problems

Open the official problem page, classify the pattern, write the invariant, and predict the complexity before coding.

#	Problem	Deliberate-practice prompt
1	**Kth Largest Element**	State the cue, invariant, edge case, and proof before implementation.
2	**Top K Frequent Elements**	State the cue, invariant, edge case, and proof before implementation.
3	**Find Median from Data Stream**	State the cue, invariant, edge case, and proof before implementation.
4	**Merge K Sorted Lists**	State the cue, invariant, edge case, and proof before implementation.
5	**Task Scheduler**	State the cue, invariant, edge case, and proof before implementation.

Before coding

+ Pattern cue
+ State
+ Invariant
+ Boundary convention

Before finishing

+ Dry run
+ Edge case
+ Time proof
+ Space proof

PUBLISHING STATUS

REVIEW GATE: STABLE TECHNICAL. AI-produced draft. A named human must verify technical accuracy, sourcing, freshness, and editorial quality before publication. Reviewed-through date: 2026-07-18.